

Aurora 战队雷达站传承文档

RoboMaster RMUC 2026 — 从 T-DT 开源到赛场应用

赵书彤

QQ: 19732579198

2026 年 9 月

摘要

本文档系统记录 2026 赛季基于东北大学 T-DT 实验室 2025 赛季开源雷达代码，改造为 Aurora 战队 RoboMaster RMUC 2026 参赛系统的完整过程，内容包括：上游开源代码的功能与不足、各模块的改造内容与改造原因、赛场应用效果，以及遗留问题与 2027 赛季改进建议，供后续接手雷达站的同学参考。

目录

1 起点：T-DT 开源代码的功能与不足	1
1.1 代码来源与基本定位	1
1.2 原版代码结构	1
2 改造总览	2
3 各模块改造说明	3
3.1 相机驱动（自主开发）	3
3.2 检测模块 detect：超分、3D 框与 5 槽位输出	3
3.3 融合模块 kalman_filter：身份锁定与盲区预测	3
3.4 标定模块 calibrate：按 2026 场地重做五点流程	3
3.5 解算模块 resolve：增加 HeightGrid 地形修正	4
3.6 配准模块 localization：CPU GICP 改为 CUDA ICP	4
3.7 串口通信：自主开发（参赛核心环节）	4
3.8 可视化模块 debug_map：盲区预测点与双倍易伤自动触发	4
3.9 工具链：控制面板、录包工具与超分训练	4
4 赛场应用效果与遗留问题	5
4.1 系统运行情况	5
4.2 双倍易伤未能触发的原因分析	5
4.3 赛事启动故障及对策	5
5 2027 赛季注意事项	5

1 起点：T-DT 开源代码的功能与不足	2
A 关键文件索引	6
B 话题速查	7

1 起点：T-DT 开源代码的功能与不足

1.1 代码来源与基本定位

上游代码为东北大学 T-DT 实验室发布的 2025 赛季雷达站开源项目(T-DT_Radar-main, MIT 协议, 技术报告见 <https://bbs.robomaster.com/article/803954>)。该项目 README 中明确说明: ” 本项目提供了除串口、相机驱动、模型训练外雷达站的全部功能”。也就是说, 感知、融合、标定与地图可视化部分是现成的, 但与裁判系统的串口通信、相机图像采集与推理模型三部分缺失, 而这三部分是实际参赛不可缺少的环节。

原版硬件平台为 Livox Avia 激光雷达、海康 CH-120-10UC 相机、i7-12700KF 处理器与 RTX A4000×2 显卡。本战队硬件为 Livox Mid-70、海康 6MP 相机、i7 处理器与 RTX 3050M 显卡, 算力明显低于原版平台, 这一差异直接决定了后续推理方案的设计思路。

1.2 原版代码结构

- src/tdt_vision/: 相机标定 (calibrate)、目标检测 (detect)、坐标解算 (resolve)、推理加速 (infer)、通用工具 (utils)
- src/lidar/: 点云配准 (localization, CPU GICP)、欧式聚类 (cluster)、动态点提取 (dynamic_cloud)
- src/fusion/: 卡尔曼融合跟踪 (kalman_filter)、地图可视化 (debug_map)
- src/interface/: 自定义消息定义 (DetectResult、MatchInfo、Radar2Sentry 等)
- src/livox_driver/: Livox 雷达驱动 (本项目直接沿用)
- src/utils/: rosbag 回放工具 (rosbag_player)、模型格式转换脚本 (onnx2trt.py)

原版代码模块划分清晰、耦合度低, 具备良好的可扩展性。但各模块实现较为基础: detect.cpp 仅 334 行、kalman_filter.cpp 仅 206 行、debug_map.cpp 仅 285 行、resolve.cpp 仅 141 行、localization.cpp 仅 187 行。核心逻辑虽然完整, 但整体只满足实验室环境下的回放验证, 与实际参赛要求存在较大差距。

2 改造总览

文件	T-DT 原版	本项目	主要改动	改动原因
detect.cpp	334	999	增加 SR 超分、3D 框、5 槽位输出、机器人号映射	算力受限，需超分提升精度；按规则要求输出指定槽位
kalman_filter.cpp	206	1488	增加 Hungarian 匹配、id_score 身份置信、行为预测	原版跟踪逻辑较简单，比赛需要持续锁定目标并支持盲区预测
debug_map.cpp	285	728	增加盲区预测点、双倍易伤自动触发	原版仅作显示，比赛需要自动触发双倍易伤
resolve.cpp	141	329	增加 HeightGrid 地形高度修正	场地存在坡道与高台，平面透视变换精度不足
localization.cpp	187	483	CPU GICP 改为 CUDA ICP	CPU 配准速度不足，改用 GPU 加速
cluster.cpp	84	250	扩充聚类与目标过滤逻辑	适配 2026 赛季车型
calibrate.cpp	190	451	标定流程按 2026 场地重写	适配新赛季场地与分色标定
完全新增的模块：				
radar_serial_node.cpp	—	新增	裁判系统串口通信 (0x0305/0x0301/0x0303)	原版缺失，参赛必需
camera/ (hik.cpp)	—	新增	海康 SDK 相机驱动	原版缺失，需自行开发
rm_control_panel	—	新增	Qt 控制面板（切换队伍、标定、录包）	原版调整配置需修改文件并重启
databag_tool	—	新增	录包工具	原版无专用录包工具
create_sr_onnx.py 等	—	新增	SR 超分模型训练与转换脚本	原版无超分方案

表 1: 从 T-DT 开源到参赛系统的核心文件改动统计。

3 各模块改造说明

3.1 相机驱动（自主开发）

原版未提供相机驱动，仅定义了 camera_image 话题，默认图像采集已由外部完成。本项目接入海康 MV-CS060 的 SDK (src/tdt_vision/camera/MV_SDK)，编写 hik.cpp 完成图像采集、曝光控制与内参数加载，并发布至 camera_image 话题。夜间场次曝光参数单独保存于 config/camera_driver_night.yaml。

3.2 检测模块 detect：超分、3D 框与 5 槽位输出

原版 334 行，仅实现 YOLO 检测与装甲板分类。由于本战队 RTX 3050M 算力明显低于原版 A4000×2，直接使用 1280 输入推理速度不足，因此采取以下措施：

1. **增加 SR 超分**: 先以较低分辨率完成快速检测, 再对感兴趣区域做超分辨率放大, 在算力受限的 GPU 上提高小目标识别率。训练与转换脚本为 `utils/create_sr_onnx.py` 与 `utils/pb2onnx_sr.py`。
2. **增加 3D 框**: 利用外参将检测框投影回三维空间, 在检测窗口中叠加蓝色长方体, 用于直观检验标定精度 (原版仅输出 2D 检测框)。
3. **增加 5 槽位输出**: 按 2026 赛季规则, 将装甲板数字映射至槽位 (1 英雄、2 工程、3 步兵三号、4 步兵四号、6 哨兵; 2026 赛季无步兵五号与空中单位), 写入 `DetectResult` 的 `red_x/blue_x` 数组。该部分存在一项重要缺陷, 详见 §4.2 (同号覆盖)。

3.3 融合模块 `kalman_filter`: 身份锁定与盲区预测

原版 206 行, 仅实现基础卡尔曼跟踪。本项目扩展至 1488 行, 主要变化:

- **Hungarian 匹配**: 对 KF 轨迹与新检测结果做二分图最优匹配, 避免同一目标匹配至多条轨迹。
- **id_score 身份置信**: 为每条轨迹维护身份置信度, 仅在装甲板数字识别结果稳定时赋值, 避免偶发误分类直接改写目标身份。
- **行为分类预测**: 目标进入盲区后保持轨迹, 依据位置与速度推断其行为类别 (CHARGING/TUNNEL/OUTPOST 四类) 并对位置进行预测, 目标重新可见后即可接管。
- **发布 /radar2sentry**: 将融合后的 5 槽位坐标发送至串口节点。

3.4 标定模块 `calibrate`: 按 2026 场地重做五点流程

在原版五点标定 (回车开始、方向键微调) 基础上扩展至 451 行: 标定点按 2026 场地更新 (我方堡垒、我方前哨、敌方基地、敌方前哨、中央高塔), 外参按红蓝双方分色保存为 `config/{red,blue}/out_matrix_*`, 并通过检测窗口中的 3D 框直观验证标定贴合程度。重要经验: 雷达每场比赛后如被搬动, 外参即失效, 局间必须重新标定。

3.5 解算模块 `resolve`: 增加 HeightGrid 地形修正

原版 141 行, 仅做平面透视变换。RMUC 场地并非平面 (存在 43 度坡与高低台), 直接投影会产生偏差。本项目增加 HeightGrid: 预先计算 `config/map/field_mesh.bin` 网格高度, 像素投影至世界坐标时查询实际地面高度加以修正, 扩展至 329 行。

3.6 配准模块 `localization`: CPU GICP 改为 CUDA ICP

原版 187 行使用 CPU GICP 配准, 本项目改为 CUDA ICP (`cuda_icp.h`), 共 483 行。原因仍是算力: CPU 配准在前数帧收敛较慢, 无法满足赛场时间要求; 改为 GPU 实现后, 前几帧即可完成对齐。场地地图 `config/map/map.pcd` 同步更换为 2026 赛季场地扫描点云。

3.7 串口通信：自主开发（参赛核心环节）

该模块为原版所没有的内容，是参赛的关键环节，也是 2026 赛季问题最集中的部分。自主编写的 `radar_serial_node.cpp` 实现以下功能：

- 0x0305 小地图上传：敌方与己方各 6 个坐标（48 字节），频率不超过 5Hz，单位为厘米，坐标置 0 表示未发送。官方标记精度规则：偏差 $<0.8\text{m}$ 计 +1, $0.8\sim1.6\text{m}$ 计 +0.5, $\geq1.6\text{m}$ 计 -0.8。
- 0x0301 与 0x0121：双倍易伤决策包（每局 2 次机会，每次 30 秒）。
- 0x0303：下行数据解析（比赛状态与血量）。
- 0x0001/0003/0101/0105/020C/020E 汇总发布至 `/match_info`：其中 0x020C 为标记成功位，0x020E 的 bit0-1 为双倍易伤机会数、bit2 为激活状态。

串口节点本身只负责数据转发，实际发送的车辆数量完全由上游 `detect` 与 `kalman` 模块决定。2026 赛季的主要问题即出在数据上游，详见 §4。

3.8 可视化模块 `debug_map`：盲区预测点与双倍易伤自动触发

原版 285 行，仅作地图显示。本项目扩展至 728 行，增加：

- 盲区预测点：目标进入盲区时，在地图上按行为分类显示预测位置。
- 双倍易伤自动触发：当机会数大于已触发数、场上存在有效标记且当前未处于激活状态时，自动发送 `/radar/decision_request`；同时保留手动方式（小地图窗口按 `v` 键）作为备份。

3.9 工具链：控制面板、录包工具与超分训练

原版仅提供 `roslaunch` 回放与 `onnx2trt` 转换脚本。本项目新增：

- `rm_control_panel` (Qt 控制面板)：比赛期间切换队伍颜色、触发标定、开关录包，无需修改配置文件并重启。
- `databag_tool`：专用录包工具，按场次自动命名并保存至 `bags/` 目录。
- `create_sr_onnx.py` 与 `pb2onnx_sr.py`：SR 超分模型的训练与格式转换。

4 赛场应用效果与遗留问题

4.1 系统运行情况

2026 区域赛局均易伤约 475 秒/局，处于较低水平。回放验证表明，识别与定位功能本身精度良好，问题不在感知环节，而在发送给裁判系统的数据内容。使用 `tools/analyze_vuln_bag.py` 对比赛回放包解码统计，结果如下：

对局	平均发送车辆数	发送 ≥ 3 辆占比	标记成功次数 (0x020C)	双倍易伤激活时长
浙江大学一队 (红方)	2.26	29%	0 次	0 秒
浙江大学二队 (红方)	3.23	68%	0 次	0 秒
中国石油大学 (红方)	1.20	0%	0 次	0 秒

表 2: 区域赛回放包统计结果。整个赛季双倍易伤均未能触发。

4.2 双倍易伤未能触发的原因分析

原因链条如下:

1. detect 模块填写槽位时, 若两辆车得到相同编号, 后写入的数据直接覆盖先写入的数据, 且无置信度仲裁 (即 §3.2 所述缺陷);
2. 车辆可见但未识别出装甲板数字时, 编号置为 -1 , 该车整帧不发送;
3. 上述两条导致每帧仅发送 1~3 辆车 (对浙江大学一队一局统计, 平均 2.26 辆, 99% 的时间不超过 3 辆);
4. 已发送坐标存在精度不足问题, 按官方规则偏差 $\geq 1.6\text{m}$ 时倒扣分数;
5. 整场比赛 0x020C 标记成功位从未出现上升沿;
6. 0x020E 机会数始终为 0, 自动触发条件永远无法满足, 双倍易伤激活时长为 0 秒。

根本原因并非双倍易伤触发逻辑本身, 而是发送车辆数量不足与坐标精度不足, 导致标记机制未能有效发挥作用。

4.3 赛事启动故障及对策

区域赛中有一局雷达未能正常启动, 双倍易伤自然全程未能触发。原因在于 3 分钟准备时间内接线与配置未能完成, 且缺少强制性的赛前自检环节。相应对策已写入 docs/preflight_check.md (赛前六项验证清单)。

5 2027 赛季注意事项

1. **车型变化:** 2027 赛季机器人数量减少一个。需按新赛季规则手册核对并修改: robot_slots.h 槽位表、radar_serial_node.cpp 协议映射、armor_class_to_slot 映射、0x0003 血量表下标。修改后必须进行全链路回放回归测试。
2. **装甲板样式变化:** 若 2027 赛季装甲板样式有变化, 现有 classify.onnx 与 armor_yolo 模型可能无法直接适用 (训练数据不包含新样式), 应尽早收集相关数据并重新训练。
3. **优先修复项:** 一是修复槽位同号覆盖问题; 二是将发送数据源改为 KF 轨迹直接输出, 不再依赖装甲板数字识别结果; 三是增加置信度门控, 坐标偏差过大时置零不发送 (协议中坐标置 0 即视为未发送)。

4. **流程规范**: 赛前必须完整执行 docs/preflight_check.md 检查清单; 并使用 tools/analyze_vuln_bag.py 对试运行回放进行统计, 平均发送车辆数 ≥ 3.5 、发送 ≥ 3 辆占比 $\geq 80\%$ 达标后方可上场。

A 关键文件索引

文件	说明 (相对 T-DT 原版的改动)
src/tdt_vision/camera/hik.cpp	新增 : 海康相机驱动
src/tdt_vision/detect/src/detect.cpp	增加超分、3D 框、5 槽输出 (334 行 $\rightarrow$ 999 行)
src/tdt_vision/resolve/src/resolve.cpp	增加 HeightGrid 地形修正 (141 行 $\rightarrow$ 329 行)
src/tdt_vision/calibrate/src/calibrate.cpp	五点标定流程重做 (190 行 $\rightarrow$ 451 行)
src/lidar/localization/src/localization.cpp	CPD GICP 改为 CUDA ICP (187 行 $\rightarrow$ 483 行)
src/lidar/cluster/src/cluster.cpp	扩充聚类与过滤逻辑 (84 行 $\rightarrow$ 250 行)
src/fusion/kalman_filter/src/kalman_filter.cpp	增加 Hungarian 匹配、身份置信、行为预测 (206 行 $\rightarrow$ 1488 行)
src/fusion/kalman_filter/src/radar_serial_model.cpp	新增 : 裁判系统串口通信
src/fusion/debug_map/debug_map.cpp	增加盲区预测、双倍易伤自动触发 (285 行 $\rightarrow$ 728 行)
src/utils/rm_control_panel/	新增 : Qt 控制面板
src/utils/databag_tool/	新增 : 专用录包工具
tools/analyze_vuln_bag.py	新增 : 回放易伤统计工具
docs/preflight_check.md	赛前六项验证清单
README_DETAIL.md	启动、接线、标定与比赛日流程
config/radar_runtime.yaml	队伍颜色、串口与回放配置入口

B 话题速查

话题	类型	说明
camera_image	Image	相机原图 (原版无驱动, 本项目新增)
/detect_result	DetectResult	YOLO 检测、装甲板分类与 5 槽位
/resolve_result	DetectResult	像素至世界坐标解算、HeightGrid 修正
/kalman_detect	DetectResult	KF 融合输出
/radar2sentry	Radar2Sentry	串口主链输入 (5 槽位敌我坐标)
/match_info	MatchInfo	比赛状态 (队伍颜色、标记、易伤、血量)
/radar/decision_request	UInt8	双倍易伤触发计数
/radar/tx_raw	UInt8MultiArray	串口发送帧回放 (调试用)